

DDL Commands for all Tables:

```
CREATE TABLE Category (  
    categoryID INT PRIMARY KEY,  
    categoryName VARCHAR(255) NOT NULL  
);
```

```
mysql> DESCRIBE Category;  
+-----+-----+-----+-----+-----+-----+  
| Field      | Type      | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| categoryID | int       | NO   | PRI | NULL    |       |  
| categoryName | varchar(255) | NO   |     | NULL    |       |  
+-----+-----+-----+-----+-----+-----+  
2 rows in set (0.00 sec)
```

```
CREATE TABLE Game (  
    gameID INT PRIMARY KEY,  
    gameDate DATETIME NOT NULL  
);
```

```
mysql> DESCRIBE Game;  
+-----+-----+-----+-----+-----+-----+  
| Field      | Type      | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| gameID     | int       | NO   | PRI | NULL    |       |  
| gameDate   | datetime  | NO   |     | NULL    |       |  
+-----+-----+-----+-----+-----+-----+  
2 rows in set (0.01 sec)
```

```
mysql> SELECT COUNT(*) FROM Game;  
+-----+  
| COUNT(*) |  
+-----+  
|      1000 |  
+-----+  
1 row in set (0.00 sec)
```

```
CREATE TABLE GameSettings (  
    categoryID INT,  
    gameID INT,  
    FOREIGN KEY (categoryID) REFERENCES Category(categoryID),  
    FOREIGN KEY (gameID) REFERENCES Game(gameID) ON DELETE CASCADE  
);
```

```
mysql> DESCRIBE GameSettings;
+-----+-----+-----+-----+-----+-----+
| Field      | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| categoryID | int  | YES  | MUL | NULL    |       |
| gameID     | int  | YES  | MUL | NULL    |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)
```

```
CREATE TABLE User (
    username VARCHAR(255) PRIMARY KEY,
    password VARCHAR(255) NOT NULL,
    fruitIcon VARCHAR(255),
    iconBackgroundColor VARCHAR(255)
);
```

```
mysql> DESCRIBE User;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| username   | varchar(255) | NO   | PRI | NULL    |       |
| password   | varchar(255) | NO   |     | NULL    |       |
| fruitIcon  | varchar(255) | YES  |     | NULL    |       |
| iconBackgroundColor | varchar(255) | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

```
mysql> SELECT COUNT(*) FROM User;
+-----+
| COUNT(*) |
+-----+
|      1000 |
+-----+
1 row in set (0.01 sec)
```

```
CREATE TABLE Question (
    questionID INT PRIMARY KEY,
    questionType VARCHAR(50) NOT NULL,
    questionText TEXT NOT NULL,
    categoryID INT NOT NULL,
    answerOptionID INT NOT NULL,
    FOREIGN KEY (categoryID) REFERENCES Category(categoryID)
); // Temporarily remove answerOptionID foreign key constraint (circular)
```

```
mysql> DESCRIBE Question;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| questionID     | int           | NO   | PRI | NULL    |       |
| questionType   | varchar(50)   | NO   |     | NULL    |       |
| questionText    | text          | NO   |     | NULL    |       |
| categoryID     | int           | NO   | MUL | NULL    |       |
| answerOptionID | int           | NO   | MUL | NULL    |       |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> SELECT COUNT(*) FROM Question;
+-----+
| COUNT(*) |
+-----+
|    49716 |
+-----+
1 row in set (0.01 sec)
```

```
CREATE TABLE QuestionOption (
    questionOptionID INT PRIMARY KEY,
    optionLabel VARCHAR(255) NOT NULL,
    optionValue VARCHAR(255) NOT NULL,
    questionID INT NOT NULL,
    FOREIGN KEY (questionID) REFERENCES Question(questionID) ON DELETE
    CASCADE
);
```

```
mysql> DESCRIBE QuestionOption;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| questionOptionID | int           | NO   | PRI | NULL    |       |
| optionLabel      | varchar(255)  | NO   |     | NULL    |       |
| optionValue      | varchar(255)  | NO   |     | NULL    |       |
| questionID       | int           | NO   | MUL | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> SELECT COUNT(*) FROM QuestionOption;
+-----+
| COUNT(*) |
+-----+
|   185494 |
+-----+
1 row in set (0.02 sec)
```

```
CREATE TABLE QuestionInstance (
    questionInstanceID INT PRIMARY KEY,
    createdAt DATETIME NOT NULL,
    answeredAt DATETIME NOT NULL,
```

```

    points INT,
    questionID INT NOT NULL,
    gameID INT NOT NULL,
    username VARCHAR(255) NOT NULL,
    userAnswer INT NOT NULL,
    FOREIGN KEY (questionID) REFERENCES Question(questionID) ON DELETE
CASCADE,
    FOREIGN KEY (gameID) REFERENCES Game(gameID) ON DELETE CASCADE,
    FOREIGN KEY (username) REFERENCES User(username) ON DELETE CASCADE,
    FOREIGN KEY (userAnswer) REFERENCES QuestionOption(questionOptionID)
ON DELETE CASCADE
);

```

```

mysql> DESCRIBE QuestionInstance;
+-----+-----+-----+-----+-----+-----+
| Field          | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| questionInstanceID | int       | NO   | PRI | NULL    |       |
| createdAt        | datetime  | NO   |     | NULL    |       |
| answeredAt       | datetime  | NO   |     | NULL    |       |
| points           | int       | YES  |     | NULL    |       |
| questionID       | int       | NO   | MUL | NULL    |       |
| gameID           | int       | NO   | MUL | NULL    |       |
| username         | varchar(255) | NO   | MUL | NULL    |       |
| userAnswer       | int       | NO   | MUL | NULL    |       |
+-----+-----+-----+-----+-----+-----+
8 rows in set (0.01 sec)

```

```

mysql> SELECT COUNT(*) FROM QuestionInstance;
+-----+
| COUNT(*) |
+-----+
| 121616 |
+-----+
1 row in set (0.01 sec)

```

```

CREATE TABLE GamePlayers (
    username VARCHAR(255),
    gameID INT,
    FOREIGN KEY (username) REFERENCES User(username) ON DELETE CASCADE,
    FOREIGN KEY (gameID) REFERENCES Game(gameID) ON DELETE CASCADE
);

```

```
mysql> DESCRIBE GamePlayers;
```

Field	Type	Null	Key	Default	Extra
username	varchar(255)	YES	MUL	NULL	
gameID	int	YES	MUL	NULL	

2 rows in set (0.00 sec)

```
mysql> SELECT COUNT(*) FROM GamePlayers;
```

COUNT(*)
6083

1 row in set (0.01 sec)

Add Back Foreign Key Constraint to Question (circular foreign key)

```
ALTER TABLE Question
```

```
ADD FOREIGN KEY (answerOptionID)
```

```
REFERENCES QuestionOption (questionOptionID)
```

```
ON DELETE CASCADE;
```

Data Parser + DB Seed Script:

The trivia question dataset parsing script and random data generation seed script can be viewed in the ***database_seeding*** folder of the Git Repository.

https://github.com/cs411-alawini/sp25-cs411-team086-CacheMoney/tree/main/database_seeding

Advanced SQL Queries and Indexing:

1. User Category Statistics

Query returns a player's accuracy for every category they've answered questions for, along with the number of correct answers (correct_count) and the number of total answers (total_count). This query is relevant since players can see which categories they have the highest/lowest accuracy in, helping them with what they need to improve.

```
SELECT
  c.categoryID,
  c.categoryName,
  (correct_count/total_count) AS accuracy,
  correct_count,
  total_count
FROM (
  SELECT
    q.categoryID,
    COUNT(*) AS correct_count
  FROM QuestionInstance qi
  NATURAL JOIN Question q
  WHERE qi.username = [username]
  AND qi.userAnswer = q.answerOptionID
  GROUP BY q.categoryID
) correctAnswers
NATURAL JOIN (
  SELECT
    q.categoryID,
    COUNT(*) AS total_count
  FROM QuestionInstance qi
  NATURAL JOIN Question q
  WHERE qi.username = [username]
  GROUP BY q.categoryID
  HAVING total_count > 0
) totalAnswers
JOIN Category c ON c.categoryID = correctAnswers.categoryID
ORDER BY accuracy DESC
LIMIT 15;
```

Example output with [username] = "user-3"

	categoryID	categoryName	accuracy	correct_co...	total_cou...	
▶	18	Movies	1.0000	9	9	
	5	Humanities	1.0000	1	1	
	3	Television	0.8750	7	8	
	14	Rated	0.8000	8	10	
	16	People	0.5000	1	2	
	6	General	0.3636	4	11	
	9	For Kids	0.3333	1	3	
	17	History	0.3333	1	3	
	10	World	0.2857	2	7	
	15	Animals	0.2500	1	4	
	12	Newest	0.1429	1	7	
	13	Sports	0.1250	1	8	
	0	Music	0.1176	2	17	

Output has less than 15 rows since "user-3" has not answered questions for more than 15 categories.

Index analysis:

Before:

```
| -> Limit: 15 row(s) (actual time=0.624..0.625 rows=11 loops=1)
|   -> Sort: accuracy DESC, limit input to 15 row(s) per chunk (actual time=0.624..0.624 rows=11 loops=1)
|     -> Stream results (cost=16.8 rows=0) (actual time=0.589..0.612 rows=11 loops=1)
|       -> Nested loop inner join (cost=16.8 rows=0) (actual time=0.582..0.602 rows=11 loops=1)
|         -> Nested loop inner join (cost=3.75 rows=0) (actual time=0.366..0.378 rows=11 loops=1)
|           -> Table scan on correctAnswers (cost=2.5..2.5 rows=0) (actual time=0.261..0.262 rows=11 loops=1)
|             -> Materialize (cost=0..0 rows=0) (actual time=0.36..0.36 rows=11 loops=1)
|               -> Table scan on <temporary> (actual time=0.35..0.351 rows=11 loops=1)
|                 -> Aggregate using temporary table (actual time=0.349..0.349 rows=11 loops=1)
|                   -> Nested loop inner join (cost=72.8 rows=3.2) (actual time=0.15..0.336 rows=28 loops=1)
|                     -> Index lookup on qi using username (username='user-3') (cost=36.4 rows=104) (actual time=0.133..0.164 rows=104 loops=1)
|                       -> Filter: (q.answerOptionID = qi.userAnswer) (cost=0.25 rows=0.05) (actual time=0.00153..0.00155 rows=0.269 loops=104)
|                         -> Single-row index lookup on q using PRIMARY (questionID=qi.questionID) (cost=0.25 rows=1) (actual time=0.00137..0.00139 rows=1 loops=104)
|                   -> Single-row index lookup on c using PRIMARY (categoryID=correctAnswers.categoryID) (cost=0.27 rows=1) (actual time=0.00128..0.0013 rows=1 loops=11)
|                 -> Index lookup on totalAnswers using <auto_key> (categoryID=correctAnswers.categoryID) (cost=0.27..2.81 rows=10.4) (actual time=0.02..0.0201 rows=1 loops=11)
|                   -> Materialize (cost=0..0 rows=0) (actual time=0.215..0.215 rows=18 loops=1)
|                     -> Filter: (total_count > 0) (actual time=0.205..0.207 rows=18 loops=1)
|                       -> Table scan on <temporary> (actual time=0.204..0.205 rows=18 loops=1)
|                         -> Aggregate using temporary table (actual time=0.204..0.204 rows=18 loops=1)
|                           -> Nested loop inner join (cost=72.8 rows=104) (actual time=0.092..0.182 rows=104 loops=1)
|                             -> Index lookup on qi using username (username='user-3') (cost=36.4 rows=104) (actual time=0.0889..0.103 rows=104 loops=1)
|                               -> Single-row index lookup on q using PRIMARY (questionID=qi.questionID) (cost=0.251 rows=1) (actual time=620e-6..640e-6 rows=1 loops=104)
```

Index scheme 1:

CREATE INDEX user_answer_name_idx ON QuestionInstance(username, userAnswer);

After:

```
| -> Limit: 15 row(s) (actual time=0.625..0.626 rows=11 loops=1)
|   -> Sort: accuracy DESC, limit input to 15 row(s) per chunk (actual time=0.624..0.625 rows=11 loops=1)
|     -> Stream results (cost=16.8 rows=0) (actual time=0.589..0.613 rows=11 loops=1)
|       -> Nested loop inner join (cost=16.8 rows=0) (actual time=0.584..0.603 rows=11 loops=1)
|         -> Nested loop inner join (cost=3.75 rows=0) (actual time=0.348..0.36 rows=11 loops=1)
|           -> Table scan on correctAnswers (cost=2.5..2.5 rows=0) (actual time=0.344..0.345 rows=11 loops=1)
|             -> Materialize (cost=0..0 rows=0) (actual time=0.344..0.344 rows=11 loops=1)
|               -> Table scan on <temporary> (actual time=0.328..0.33 rows=11 loops=1)
|                 -> Aggregate using temporary table (actual time=0.328..0.328 rows=11 loops=1)
|                   -> Nested loop inner join (cost=72.8 rows=5.2) (actual time=0.153..0.315 rows=28 loops=1)
|                     -> Index lookup on qi using user_answer_name_idx (username='user-3') (cost=36.4 rows=104) (actual time=0.139..0.156 rows=104 loops=1)
|                       -> Filter: (q.answerOptionID = qi.userAnswer) (cost=0.25 rows=0.05) (actual time=0.00142..0.00143 rows=0.269 loops=104)
|                         -> Single-row index lookup on q using PRIMARY (questionID=qi.questionID) (cost=0.25 rows=1) (actual time=0.00126..0.00129 rows=1 loops=104)
|                   -> Single-row index lookup on c using PRIMARY (categoryID=correctAnswers.categoryID) (cost=0.27 rows=1) (actual time=0.00118..0.0012 rows=1 loops=11)
|                 -> Index lookup on totalAnswers using <auto_key> (categoryID=correctAnswers.categoryID) (cost=0.27..2.81 rows=10.4) (actual time=0.0217..0.0219 rows=1 loops=11)
|                   -> Materialize (cost=0..0 rows=0) (actual time=0.234..0.234 rows=18 loops=1)
|                     -> Filter: (total_count > 0) (actual time=0.224..0.226 rows=18 loops=1)
|                       -> Table scan on <temporary> (actual time=0.223..0.223 rows=18 loops=1)
|                         -> Aggregate using temporary table (actual time=0.223..0.223 rows=18 loops=1)
|                           -> Nested loop inner join (cost=72.8 rows=104) (actual time=0.112..0.201 rows=104 loops=1)
|                             -> Index lookup on qi using user_answer_name_idx (username='user-3') (cost=36.4 rows=104) (actual time=0.11..0.124 rows=104 loops=1)
|                               -> Single-row index lookup on q using PRIMARY (questionID=qi.questionID) (cost=0.251 rows=1) (actual time=597e-6..617e-6 rows=1 loops=104)
```

Explanation:

Adding an index on *QuestionInstance(username, userAnswer)* did **not** have an effect on the cost of the query. This is likely because the question instance *username* and *userAnswer* are not used together, with both the *username* and *userAnswer* being compared to different, exact matches. Because *username* and *userAnswer* are foreign keys in *QuestionInstance*, MySQL already creates indexes on *username* and *userAnswer* which provide more benefit than creating a shared index on both fields.

Index scheme 2:

CREATE INDEX category_answer_idx ON Question(categoryID, answerOptionID);

After:

```
| -> Limit: 15 row(s) (actual time=0.599..0.6 rows=11 loops=1)
|   -> Sort: accuracy DESC, limit input to 15 row(s) per chunk (actual time=0.598..0.599 rows=11 loops=1)
|     -> Stream results (cost=16.8 rows=0) (actual time=0.551..0.587 rows=11 loops=1)
|       -> Nested loop inner join (cost=16.8 rows=0) (actual time=0.346..0.379 rows=11 loops=1)
|         -> Nested loop inner join (cost=3.75 rows=0) (actual time=0.326..0.35 rows=11 loops=1)
|           -> Table scan on correctAnswers (cost=2.5..2.5 rows=0) (actual time=0.322..0.324 rows=11 loops=1)
|             -> Materialize (cost=0..0 rows=0) (actual time=0.322..0.322 rows=11 loops=1)
|               -> Table scan on <temporary> (actual time=0.313..0.314 rows=11 loops=1)
|                 -> Aggregate using temporary table (actual time=0.312..0.312 rows=11 loops=1)
|                   -> Nested loop inner join (cost=72.8 rows=5.2) (actual time=0.136..0.33 rows=28 loops=1)
|                     -> Index lookup on qi using username (username='user-3') (cost=36.4 rows=104) (actual time=0.122..0.139 rows=104 loops=1)
|                       -> Filter: (q.answerOptionID = qi.userAnswer) (cost=0.25 rows=0.05) (actual time=0.00149..0.00149 rows=0.269 loops=104)
|                         -> Single-row index lookup on q using PRIMARY (questionID=qi.questionID) (cost=0.25 rows=1) (actual time=0.00128..0.0013 rows=1 loops=104)
|                   -> Single-row index lookup on c using PRIMARY (categoryID=correctAnswers.categoryID) (cost=0.27 rows=1) (actual time=0.00219..0.00221 rows=1 loops=11)
|                 -> Index lookup on totalAnswers using <auto_key> (categoryID=correctAnswers.categoryID) (cost=0.27..2.81 rows=10.4) (actual time=0.0203..0.0204 rows=1 loops=11)
|                   -> Materialize (cost=0..0 rows=0) (actual time=0.218..0.218 rows=18 loops=1)
|                     -> Filter: (total_count > 0) (actual time=0.208..0.211 rows=18 loops=1)
|                       -> Table scan on <temporary> (actual time=0.208..0.208 rows=18 loops=1)
|                         -> Aggregate using temporary table (actual time=0.207..0.207 rows=18 loops=1)
|                           -> Nested loop inner join (cost=72.8 rows=104) (actual time=0.0846..0.187 rows=104 loops=1)
|                             -> Index lookup on qi using username (username='user-3') (cost=36.4 rows=104) (actual time=0.084..0.0983 rows=104 loops=1)
|                               -> Single-row index lookup on q using PRIMARY (questionID=qi.questionID) (cost=0.251 rows=1) (actual time=700e-6..719e-6 rows=1 loops=104)
```

Explanation:

Adding an index on *Question(categoryID, answerOptionID)* did **not** have an effect on the cost of the query. This is likely because the question *categoryID* and *answerOptionID* are not used together, with the *categoryID* being used for GROUP BY and *answerOptionID* being used to filter out rows before the GROUP BY. Because *categoryID* and *answerOptionID* are foreign keys in *Question*, MySQL already creates indexes on *categoryID* and *answerOptionID* which provide more benefit than creating a shared index on both fields.

Index scheme 3:

CREATE INDEX cat_idx ON Category(category_ID, categoryName);

After:

```
--> Limit: 15 row(s) (actual time=0.398..0.599 rows=11 loops=1)
--> Sort: accuracy DESC, limit input to 15 row(s) per chunk (actual time=0.597..0.598 rows=11 loops=1)
--> Stream aggregate (cost=16.8 rows=0) (actual time=0.463..0.587 rows=11 loops=1)
--> Nested loop inner join (cost=16.8 rows=0) (actual time=0.558..0.578 rows=11 loops=1)
--> Materialize (cost=0.0 rows=0) (actual time=0.355..0.362 rows=11 loops=1)
--> Table scan on correctAnswers (cost=2.3..2.3 rows=0) (actual time=0.346..0.347 rows=11 loops=1)
--> Materialize (cost=0.0 rows=0) (actual time=0.345..0.345 rows=11 loops=1)
--> Table scan on category (cost=0.3..0.3 rows=11 loops=1)
--> Aggregate using temporary table (actual time=0.333..0.337 rows=11 loops=1)
--> Nested loop inner join (cost=72.8 rows=21) (actual time=0.338..0.342 rows=28 loops=1)
--> Index lookup on q1 using username (username='user-3') (cost=36.4 rows=104) (actual time=0.145..0.162 rows=104 loops=1)
--> Filter: (q.answerOptionID = q1.answerOptionID) (cost=0.23 rows=0) (actual time=0.201..0.201 rows=263 loops=104)
--> Single-row index lookup on q using PRIMARY (questionID=q1.questionID) (cost=0.23 rows=1) (actual time=0.00125..0.00127 rows=1 loops=104)
--> Single-row index lookup on q using PRIMARY (categoryID=correctAnswers.categoryID) (cost=0.27 rows=1) (actual time=0.00114..0.00116 rows=1 loops=11)
--> Index lookup on totalAnswers using auto_key0 (categoryID=correctAnswers.categoryID) (cost=0.27..2.81 rows=10.4) (actual time=0.0192..0.0194 rows=1 loops=11)
--> Materialize (cost=0.0 rows=0) (actual time=0.227..0.228 rows=18 loops=1)
--> Filter: (total_count > 0) (actual time=0.194..0.199 rows=18 loops=1)
--> Table scan on category (actual time=0.196..0.197 rows=18 loops=1)
--> Aggregate using temporary table (actual time=0.135..0.139 rows=18 loops=1)
--> Nested loop inner join (cost=72.8 rows=104) (actual time=0.2873..0.175 rows=104 loops=1)
--> Index lookup on q1 using username (username='user-3') (cost=36.4 rows=104) (actual time=0.0852..0.0993 rows=104 loops=1)
--> Single-row index lookup on q using PRIMARY (questionID=q1.questionID) (cost=0.231 rows=1) (actual time=0.007e-6..0.007e-6 rows=1 loops=104)
```

Explanation:

Adding an index on *Category(category_ID, categoryName)* did **not** have an effect on the cost of the query. This is because the question *categoryID* and *categoryName* are not used in the cost-heavy sections of the query like GROUP BY, HAVING, or WHERE. In addition, there are only 21 categories, so any index over *Category* will likely not have a large effect.

Summary:

No index schemes were found to change the performance of the user category statistics query. This can be explained by the fact that MySQL automatically creates indexes on foreign keys, and most of the attributes used in the GROUP BY, JOIN, HAVING, and WHERE were all foreign keys.

2. Top 15 player scores

```
SELECT q.username, SUM(q.points) as totalScore
FROM QuestionInstance q NATURAL JOIN GamePlayers g
GROUP BY q.username
ORDER BY totalScore DESC
LIMIT 15;
```

This query finds the top 15 players with the highest total scores in all games, ordering them from highest to lowest scores. This can be useful for implementing a leaderboard with an updated track of the highest player scores with this information.

This uses NATURAL JOIN, allowing us to join multiple relations, as well as Aggregation via GROUP BY, making it an advanced query.

	username	totalScore
	user-424	12359
	user-687	12203
	user-386	12008
	user-469	11930
	user-319	10794
	user-227	10733
	user-973	10376
	user-949	10337
	user-465	10155
	user-438	10051
	user-854	9691
	user-487	9545
	user-482	9355
	user-514	9327
	user-996	9316

Index #1:

BEFORE:

```
| -> Limit: 15 row(s) (actual time=797..797 rows=15 loops=1)
|   -> Sort: totalScore DESC, limit input to 15 row(s) per chunk (actual time=797..797 rows=15 loops=1)
|     -> Table scan on <temporary> (actual time=797..797 rows=1000 loops=1)
|       -> Aggregate using temporary table (actual time=797..797 rows=1000 loops=1)
|         -> Nested loop inner join (cost=252821 rows=721) (actual time=0.0989..739 rows=121616 loops=1)
|           -> Filter: (g.gameID is not null) (cost=613 rows=6083) (actual time=0.0211..3.11 rows=6083 loops=1)
|             -> Table scan on g (cost=613 rows=6083) (actual time=0.0206..2.68 rows=6083 loops=1)
|           -> Filter: (q.username = g.username) (cost=29.6 rows=0.118) (actual time=0.0831..0.12 rows=20 loops=6083)
|             -> Index lookup on q using gameID (gameID=g.gameID) (cost=29.6 rows=118) (actual time=0.0605..0.105 rows=144 loops=6083)
|
```

CREATING INDEX:

```
mysql> CREATE INDEX idx_questioninstance_username ON QuestionInstance(username);
Query OK, 0 rows affected, 1 warning (0.42 sec)
Records: 0 Duplicates: 0 Warnings: 1
```

AFTER:

```
| -> Limit: 15 row(s) (actual time=792..792 rows=15 loops=1)
|   -> Sort: totalScore DESC, limit input to 15 row(s) per chunk (actual time=792..792 rows=15 loops=1)
|     -> Table scan on <temporary> (actual time=791..791 rows=1000 loops=1)
|       -> Aggregate using temporary table (actual time=791..791 rows=1000 loops=1)
|         -> Nested loop inner join (cost=252821 rows=721) (actual time=0.099..736 rows=121616 loops=1)
|           -> Filter: (g.gameID is not null) (cost=613 rows=6083) (actual time=0.0211..3.06 rows=6083 loops=1)
|             -> Table scan on g (cost=613 rows=6083) (actual time=0.0206..2.65 rows=6083 loops=1)
|           -> Filter: (q.username = g.username) (cost=29.6 rows=0.118) (actual time=0.0828..0.119 rows=20 loops=6083)
|             -> Index lookup on q using gameID (gameID=g.gameID) (cost=29.6 rows=118) (actual time=0.0603..0.104 rows=144 loops=6083)
|
```

As we can see using EXPLAIN ANALYZE, this resulted in our cost not changing, staying at **252821** before and after creating the index.

The reason I attempted to index this was because indexing username alone helps us speed up the GROUP BY operation in the query. Since we are grouping by username, the database can then quickly find and group rows by this key, which should reduce the cost of the operation.

However, as we see in the EXPLAIN ANALYZE outputs, the cost was not changed at all. This is likely because the query already performs a JOIN and GROUP BY based on username and gameID for grouping and joining, so just indexing username doesn't optimize it enough, which is why there is no cost improvement.

Index #2:

BEFORE:

```
| -> Limit: 15 row(s) (actual time=797..797 rows=15 loops=1)
|   -> Sort: totalScore DESC, limit input to 15 row(s) per chunk (actual time=797..797 rows=15 loops=1)
|     -> Table scan on <temporary> (actual time=797..797 rows=1000 loops=1)
|       -> Aggregate using temporary table (actual time=797..797 rows=1000 loops=1)
|         -> Nested loop inner join (cost=252821 rows=721) (actual time=0.0989..739 rows=121616 loops=1)
|           -> Filter: (g.gameID is not null) (cost=613 rows=6083) (actual time=0.0211..3.11 rows=6083 loops=1)
|             -> Table scan on g (cost=613 rows=6083) (actual time=0.0206..2.68 rows=6083 loops=1)
|           -> Filter: (q.username = g.username) (cost=29.6 rows=0.118) (actual time=0.0831..0.12 rows=20 loops=6083)
|             -> Index lookup on q using gameID (gameID=g.gameID) (cost=29.6 rows=118) (actual time=0.0605..0.105 rows=144 loops=6083)
|
```

CREATING INDEX:

```
mysql> CREATE INDEX idx_questioninstance_gameID_username ON QuestionInstance(gameID, username);
Query OK, 0 rows affected (0.40 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

AFTER:

```
| -> Limit: 15 row(s) (actual time=231..231 rows=15 loops=1)
|   -> Sort: totalScore DESC, limit input to 15 row(s) per chunk (actual time=231..231 rows=15 loops=1)
|     -> Table scan on <temporary> (actual time=230..230 rows=1000 loops=1)
|       -> Aggregate using temporary table (actual time=230..230 rows=1000 loops=1)
|         -> Nested loop inner join (cost=41816 rows=117726) (actual time=0.589..176 rows=121616 loops=1)
|           -> Filter: ((g.gameID is not null) and (g.username is not null)) (cost=613 rows=6083) (actual time=0.0194..3.05 rows=6083 loops=1)
|             -> Table scan on g (cost=613 rows=6083) (actual time=0.0185..2.58 rows=6083 loops=1)
|           -> Index lookup on q using idx_questioninstance_gameID_username (gameID=g.gameID, username=g.username) (cost=4.84 rows=19.4) (actual time=0.0231..0.0275 rows=20 loops=6083)
|
```

As we can see using EXPLAIN ANALYZE, this resulted in our cost changing from **252821** to **41816**.

Indexing this way lets us quickly find the relevant rows without scanning the whole table. The two main operations in the query are to JOIN on gameID and GROUP BY username, so creating an index on these two fields results in more efficient filtering. The index helps look up matching gameID values and filter by usernames if the gameID is queried first. And as we can see, it improves performance, dropping the cost from 252821 to 41816.

Index #3:

BEFORE:

```
| -> Limit: 15 row(s) (actual time=791..791 rows=15 loops=1)
|   -> Sort: totalScore DESC, limit input to 15 row(s) per chunk (actual time=791..791 rows=15 loops=1)
|     -> Table scan on <temporary> (actual time=791..791 rows=1000 loops=1)
|       -> Aggregate using temporary table (actual time=791..791 rows=1000 loops=1)
|         -> Nested loop inner join (cost=252821 rows=721) (actual time=0.0934..736 rows=121616 loops=1)
|           -> Filter: (g.gameID is not null) (cost=613 rows=6083) (actual time=0.0189..3.08 rows=6083 loops=1)
|             -> Table scan on g (cost=613 rows=6083) (actual time=0.0185..2.66 rows=6083 loops=1)
|           -> Filter: (q.username = g.username) (cost=29.6 rows=0.118) (actual time=0.0827..0.119 rows=20 loops=6083)
|             -> Index lookup on q using gameID (gameID=g.gameID) (cost=29.6 rows=118) (actual time=0.0602..0.104 rows=144 loops=6083)
|
```

CREATING INDEX:

```
mysql> CREATE INDEX idx_questioninstance_username_gameid_points ON QuestionInstance(username, gameID, points);
Query OK, 0 rows affected (0.53 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

AFTER:

```
| -> Limit: 15 row(s) (actual time=181..181 rows=15 loops=1)
|   -> Sort: totalScore DESC, limit input to 15 row(s) per chunk (actual time=181..181 rows=15 loops=1)
|     -> Table scan on <temporary> (actual time=180..180 rows=1000 loops=1)
|       -> Aggregate using temporary table (actual time=180..180 rows=1000 loops=1)
|         -> Nested loop inner join (cost=34045 rows=124934) (actual time=0.0445..128 rows=121616 loops=1)
|           -> Filter: ((g.username is not null) and (g.gameID is not null)) (cost=613 rows=6083) (actual time=0.019..3.12 rows=6083 loops=1)
|             -> Table scan on g (cost=613 rows=6083) (actual time=0.018..2.62 rows=6083 loops=1)
|           -> Covering index lookup on q using idx_questioninstance_username_gameid_points (username=g.username, gameID=g.gameID) (cost=3.44 rows=20.5) (actual time=0.0168..0.0194 rows=20 loops=6083)
|
```

As we can see using EXPLAIN ANALYZE, this resulted in our cost changing from **252821** to **34045**.

Creating this index improves the query by optimizing the JOIN and GROUP BY statements. It speeds up the JOIN between QuestionInstance and GamePlayers by indexing the username and gameId. This helps it efficiently calculate SUM(points) in the query, reducing the amount of total table scans and lookups and successfully reducing the overall cost of the query.

3. Biggest Categories By Number of Questions:

```
SELECT Distinct categoryName, COUNT(questionID)
FROM Question NATURAL JOIN Category
GROUP BY categoryName
ORDER BY COUNT(questionID) DESC
LIMIT 15;
```

Finds the top 15 biggest categories by number of questions

Query includes JOIN relation between Question and Category tables, GROUP BY to group all results to the category it belongs to

Relevant because it helps us determine which are categories that have the most data (questions) which can help when choosing the sample of questions for users to answer

Proof (limited to 15 rows):

	categoryName	COUNT(questionID)	
	Music	5579	
	Television	5230	
	World	4875	
	Movies	4313	
	General	3290	
	Celebrities	3196	
	Newest	3016	
	Sports	2840	
	People	2743	
	Science Technology	2486	
	Rated	2185	
	History	1645	
	Animals	1366	
	Literature	1288	
	Hobbies	1242	

EXPLAIN ANALYZE:

BEFORE INDEX (for all 3 indexes):

```

-----
| -> Limit: 15 row(s) (actual time=25.5..25.5 rows=15 loops=1)
| -> Sort: COUNT(questionID) DESC, limit input to 15 row(s) per chunk (actual time=25.5..25.5 rows=15 loops=1)
| -> Table scan on <temporary> (actual time=25.4..25.5 rows=22 loops=1)
| -> Aggregate using temporary table (actual time=25.4..25.4 rows=22 loops=1)
| -> Nested loop inner join (cost=5376 rows=53014) (actual time=0.0512..10.2 rows=49716 loops=1)
| -> Covering index scan on Category using double index (cost=2.45 rows=22) (actual time=0.0222..0.0302 rows=22 loops=1)
| -> Covering index lookup on Question using categoryID (categoryID=Category.categoryID) (cost=14.2 rows=2410) (actual time=0.0115..0.358 rows=2260 loops=22)
-----

```

We found that no addition of indices would impact and improve cost. We added:

```
CREATE INDEX idx ON Question(category_ID);
CREATE INDEX idx ON Question(category_ID, questionID);
CREATE INDEX idx ON Category(category_ID, categoryName);
```

AFTER INDEX:

```

-> Limit: 15 row(s) (actual time=25.7..25.7 rows=15 loops=1)
-> Sort: 'COUNT(questionID)' DESC, limit input to 15 row(s) per chunk (actual time=25.7..25.7 rows=15 loops=1)
-> Table scan on <temporary> (actual time=25.7..25.7 rows=22 loops=1)
-> Aggregate using temporary table (actual time=25.7..25.7 rows=22 loops=1)
-> Nested loop inner join (cost=5376 rows=53014) (actual time=0.0519..10.7 rows=49716 loops=1)
-> Covering index scan on Category using double index (cost=2.45 rows=22) (actual time=0.0211..0.0282 rows=22 loops=1)
-> Covering index lookup on Question using categoryID (categoryID=Category.categoryID) (cost=14.2 rows=2410) (actual time=0.011..0.351 rows=2260 loops=22)

```

As can be seen, the cost remains at 5376. What we found was that the indices likely don't change performance as all the create statements reference categoryID which is a part of the Category table. This table houses all the question categories but only has around 21 rows. This is likely the cause of the index not changing performance as there are already less rows to go over.

4. Average Time Taken For A User Per Category

This query finds the average amount of time it took a user to answer questions in a category. This query is relevant since it helps players see what they spend the most time on and which categories they are comfortable/less comfortable with.

```
SELECT c.categoryName, qi.username, AVG(TIMESTAMPDIFF(SECOND, qi.createdAt,
qi.answeredAt)) as timeavg
FROM QuestionInstance qi
NATURAL JOIN Question q
NATURAL JOIN Category c
WHERE qi.username=[username]
GROUP BY qi.username, c.categoryName
ORDER BY timeavg
LIMIT 15;
```

Example output with [username] = "user-4"

```
mysql> SELECT c.categoryName, qi.username, AVG(TIMESTAMPDIFF(SECOND, qi.createdAt, qi.answeredAt)) as timeavg
-> FROM QuestionInstance qi
-> NATURAL JOIN Question q
-> NATURAL JOIN Category c
-> WHERE qi.username="user-4"
-> GROUP BY qi.username, c.categoryName
-> ORDER BY timeavg
-> LIMIT 15;
+-----+-----+-----+
| categoryName | username | timeavg |
+-----+-----+-----+
| People       | user-4   | 3.3333  |
| World        | user-4   | 10.6667 |
| Rated        | user-4   | 12.0000 |
| Celebrities  | user-4   | 12.7143 |
| History      | user-4   | 12.7500 |
| Newest       | user-4   | 14.1667 |
| Movies       | user-4   | 15.6667 |
| Literature   | user-4   | 15.6667 |
| Sports       | user-4   | 16.0000 |
| Animals      | user-4   | 16.0000 |
| Television   | user-4   | 17.0690 |
| General      | user-4   | 18.5000 |
| Hobbies      | user-4   | 19.0000 |
| Music        | user-4   | 19.0833 |
| Religion Faith | user-4   | 22.0000 |
+-----+-----+-----+
15 rows in set (0.01 sec)
```

Index Analysis:

Index #1:

Before indexing:

```
| -> Limit: 15 row(s) (actual time=0.55..0.551 rows=15 loops=1)
|   -> Sort: timeavg, limit input to 15 row(s) per chunk (actual time=0.549..0.55 rows=15 loops=1)
|     -> Table scan on <temporary> (actual time=0.531..0.534 rows=16 loops=1)
|       -> Aggregate using temporary table (actual time=0.531..0.53 rows=16 loops=1)
|         -> Nested loop inner join (cost=126 rows=120) (actual time=0.148..0.441 rows=120 loops=1)
|           -> Nested loop inner join (cost=84 rows=120) (actual time=0.145..0.36 rows=120 loops=1)
|             -> Index lookup on qi using username (username='user-4') (cost=42 rows=120) (actual time=0.138..0.174 rows=120 loops=1)
|               -> Single-row index lookup on q using PRIMARY (questionID=qi.questionID) (cost=0.251 rows=1) (actual time=0.00126..0.00128 rows=1 loops=120)
|             -> Single-row index lookup on c using PRIMARY (categoryID=q.categoryID) (cost=0.251 rows=1) (actual time=500e-6..520e-6 rows=1 loops=120)
```

Index Scheme:

CREATE INDEX idx ON QuestionInstance(createdAt, answeredAt);

After indexing:

```
| -> Limit: 15 row(s) (actual time=0.544..0.546 rows=15 loops=1)
|   -> Sort: timeavg, limit input to 15 row(s) per chunk (actual time=0.544..0.544 rows=15 loops=1)
|     -> Table scan on <temporary> (actual time=0.525..0.528 rows=16 loops=1)
|       -> Aggregate using temporary table (actual time=0.524..0.524 rows=16 loops=1)
|         -> Nested loop inner join (cost=126 rows=120) (actual time=0.145..0.433 rows=120 loops=1)
|           -> Nested loop inner join (cost=84 rows=120) (actual time=0.141..0.354 rows=120 loops=1)
|             -> Index lookup on qi using username (username='user-4') (cost=42 rows=120) (actual time=0.135..0.17 rows=120 loops=1)
|               -> Single-row index lookup on q using PRIMARY (questionID=qi.questionID) (cost=0.251 rows=1) (actual time=0.00139..0.00141 rows=1 loops=120)
|             -> Single-row index lookup on c using PRIMARY (categoryID=q.categoryID) (cost=0.251 rows=1) (actual time=505e-6..525e-6 rows=1 loops=120)
```

As we can see using EXPLAIN ANALYZE, this resulted in our cost not changing and staying at **126**.

Explanation: The goal of this index is to optimize calculating the time stamp difference between the two columns. However, we are joining multiple tables and grouping by username and categoryName. The indexes on createdAt and answeredAt do not help with the needs of the join and grouping operations, so it didn't really help reduce any operations, which is why the cost was unaffected.

Index #2:

Before indexing:

```
| -> Limit: 15 row(s) (actual time=0.55..0.551 rows=15 loops=1)
|   -> Sort: timeavg, limit input to 15 row(s) per chunk (actual time=0.549..0.55 rows=15 loops=1)
|     -> Table scan on <temporary> (actual time=0.531..0.534 rows=16 loops=1)
|       -> Aggregate using temporary table (actual time=0.531..0.53 rows=16 loops=1)
|         -> Nested loop inner join (cost=126 rows=120) (actual time=0.148..0.441 rows=120 loops=1)
|           -> Nested loop inner join (cost=84 rows=120) (actual time=0.145..0.36 rows=120 loops=1)
|             -> Index lookup on qi using username (username='user-4') (cost=42 rows=120) (actual time=0.138..0.174 rows=120 loops=1)
|               -> Single-row index lookup on q using PRIMARY (questionID=qi.questionID) (cost=0.251 rows=1) (actual time=0.00126..0.00128 rows=1 loops=120)
|             -> Single-row index lookup on c using PRIMARY (categoryID=q.categoryID) (cost=0.251 rows=1) (actual time=500e-6..520e-6 rows=1 loops=120)
```

Index Scheme

CREATE INDEX idx ON QuestionInstance(username, createdAt, answeredAt);

```

-> Limit: 15 row(s) (actual time=0.529..0.53 rows=15 loops=1)
-> Sort: timeavg, limit input to 15 row(s) per chunk (actual time=0.528..0.529 rows=15 loops=1)
-> Table scan on <temporary> (actual time=0.509..0.512 rows=16 loops=1)
-> Aggregate using temporary table (actual time=0.508..0.508 rows=16 loops=1)
-> Nested loop inner join (cost=126 rows=120) (actual time=0.14..0.418 rows=120 loops=1)
-> Nested loop inner join (cost=84 rows=120) (actual time=0.136..0.337 rows=120 loops=1)
-> Index lookup on q1 using username (username='user-4') (cost=42 rows=120) (actual time=0.129..0.165 rows=120 loops=1)
-> Single-row index lookup on q using PRIMARY (questionID=q1.questionID) (cost=0.251 rows=1) (actual time=0.00129..0.00131 rows=1 loops=120)
-> Single-row index lookup on c using PRIMARY (categoryID=q.categoryID) (cost=0.251 rows=1) (actual time=511e-6..532e-6 rows=1 loops=120)

```

After indexing:

```

-----+
-> Limit: 15 row(s) (actual time=0.544..0.546 rows=15 loops=1)
-> Sort: timeavg, limit input to 15 row(s) per chunk (actual time=0.544..0.544 rows=15 loops=1)
-> Table scan on <temporary> (actual time=0.525..0.528 rows=16 loops=1)
-> Aggregate using temporary table (actual time=0.524..0.524 rows=16 loops=1)
-> Nested loop inner join (cost=126 rows=120) (actual time=0.145..0.433 rows=120 loops=1)
-> Nested loop inner join (cost=84 rows=120) (actual time=0.141..0.354 rows=120 loops=1)
-> Index lookup on q1 using username (username='user-4') (cost=42 rows=120) (actual time=0.135..0.17 rows=120 loops=1)
-> Single-row index lookup on q using PRIMARY (questionID=q1.questionID) (cost=0.251 rows=1) (actual time=0.00139..0.00141 rows=1 loops=120)
-> Single-row index lookup on c using PRIMARY (categoryID=q.categoryID) (cost=0.251 rows=1) (actual time=505e-6..525e-6 rows=1 loops=120)
-----+

```

As we can see using EXPLAIN ANALYZE, this resulted in our cost not changing and staying at **126**.

Explanation:

This index does not optimize our cost as expected because even though it covers the username for grouping, it doesn't fully optimize the JOIN operations or the final calculation of the time differences. The performance of the query is more impacted by the joins with Question and Category, and this isn't included in this index making it not improve the cost.

Index #3:

Before indexing:

```

-> Limit: 15 row(s) (actual time=0.55..0.551 rows=15 loops=1)
-> Sort: timeavg, limit input to 15 row(s) per chunk (actual time=0.549..0.55 rows=15 loops=1)
-> Table scan on <temporary> (actual time=0.531..0.534 rows=16 loops=1)
-> Aggregate using temporary table (actual time=0.53..0.53 rows=16 loops=1)
-> Nested loop inner join (cost=126 rows=120) (actual time=0.148..0.441 rows=120 loops=1)
-> Nested loop inner join (cost=84 rows=120) (actual time=0.145..0.36 rows=120 loops=1)
-> Index lookup on q1 using username (username='user-4') (cost=42 rows=120) (actual time=0.138..0.174 rows=120 loops=1)
-> Single-row index lookup on q using PRIMARY (questionID=q1.questionID) (cost=0.251 rows=1) (actual time=0.00126..0.00128 rows=1 loops=120)
-> Single-row index lookup on c using PRIMARY (categoryID=q.categoryID) (cost=0.251 rows=1) (actual time=500e-6..520e-6 rows=1 loops=120)

```

Index Scheme:

CREATE INDEX idx ON Category(categoryName);

After indexing:

```

-----+
-> Limit: 15 row(s) (actual time=0.539..0.541 rows=15 loops=1)
-> Sort: timeavg, limit input to 15 row(s) per chunk (actual time=0.538..0.539 rows=15 loops=1)
-> Table scan on <temporary> (actual time=0.507..0.524 rows=16 loops=1)
-> Aggregate using temporary table (actual time=0.506..0.506 rows=16 loops=1)
-> Nested loop inner join (cost=126 rows=120) (actual time=0.136..0.417 rows=120 loops=1)
-> Nested loop inner join (cost=84 rows=120) (actual time=0.132..0.336 rows=120 loops=1)
-> Index lookup on q1 using username (username='user-4') (cost=42 rows=120) (actual time=0.126..0.16 rows=120 loops=1)
-> Single-row index lookup on q using PRIMARY (questionID=q1.questionID) (cost=0.251 rows=1) (actual time=0.00133..0.00135 rows=1 loops=120)
-> Single-row index lookup on c using PRIMARY (categoryID=q.categoryID) (cost=0.251 rows=1) (actual time=511e-6..532e-6 rows=1 loops=120)
-----+

```

Explanation:

The goal of this index is to optimize the GROUP BY c.categoryName. However, this index did not have an effect on the cost. This is likely because there are only 21 categories, so indexing categories does not help very much. In addition, there is only one category name for every category ID, so the existing index on categoryID might already optimize the query.